Міністерство освіти і науки України
Національний університет «Одеська політехніка»
Навчально-науковий інститут комп’ютерних систем
Кафедра інформаційних систем

Лабораторна робота № 2
з дисципліни: «Теорія алгоритмів»
Тема: «Логарифмічні алгоритми сортування»

Варіант № 2

Виконав:
Студент групи АІ-254
Гармаш Г.Є.
Перевірили:
Смик С. Ю.
Арсірій О.О.

Одеса 2026

Мета роботи: Метою виконання лабораторної роботи є набуття практичних навичок із проектування, реалізації, тестування та аналізу алгоритмів сортування злиттям ітеративної та рекурсивної версії, а також рекурсивного швидкого сортування. За рахунок виконання трасування та візуалізації у вигляді графа. Студент має практично довести логарифмічну часову складність та стійкість або нестійкість вивчених алгоритмів, а також порівняти ці алгоритми за кількістю базових операцій.

Завдання:

1. Спроектувати, реалізувати, протестувати та проаналізувати алгоритм сортування злиттям.
2. Спроектувати, реалізувати, протестувати та проаналізувати алгоритм швидкого сортування
3. Порівняти обчислювальну складність алгоритмів

2	79, 97, 82, 18, 20, 2, 88, 61, 17

Результати виконання завдання:

1. **Сортування злиттям**

Таблиця 2.1 – Псевдокоди ітеративного та рекурсивного алгоритму сортування злиттям

Ітеративний алгоритм	Рекурсивний алгоритм
function mergeSortIterative(a : int[n]): for i = 1 to n, i *= 2 for j = 0 to n - i, j += 2 * i merge_iterative (a, j, j + i, min(j + 2 * i, n))	MergeSortRecursive (A) n ← length(A) if n ≤ 1 then return A mid ← n / 2 left_half ← A[0...mid-1] // Розділяємо масив right_half ← A[mid...n-1] sorted_left ← MergeSortRecursive (left_half) sorted_right ← MergeSortRecursive(right_half) return Merge(sorted_left, sorted_right) // Зливаємо відсортовані половини
// Допоміжна функція для злиття function merge_iterative (a : int[n]; left, mid, right : int): it1 = 0 it2 = 0 result : int[right - left] while left + it1 < mid and mid + it2 < right if a[left + it1] < a[mid + it2] result[it1 + it2] = a[left + it1] it1 += 1 else result[it1 + it2] = a[mid + it2] it2 += 1	// Допоміжна функція для злиття Merge(Left, Right): Result ← [] i ← 0 j ← 0 while i < length(Left) and j < length(Right) do if Left[i] ≤ Right[j] then append Left[i] to Result i ← i + 1 else append Right[j] to Result j ← j + 1

while left + it1 < mid result[it1 + it2] = a[left + it1] it1 += 1 while mid + it2 < right result[it1 + it2] = a[mid + it2] it2 += 1 for i = 0 to it1 + it2 a[left + i] = result[i]	while i < length(Left) do // Додаємо елементи, що залишилися append Left[i] to Result i ← i + 1 while j < length(Right) do append Right[j] to Result j ← j + 1 return Result

Лістинг 2.1 – Python-код ітеративної реалізації алгоритму сортування злиттям.

```
def merge_sort_iterative(a):
    n = len(a)
    comparisons = 0
    assignments = 0
    i = 1

    while i < n:
        j = 0
        while j < n - i:
            left = j
            mid = j + i
            right = min(j + 2 * i, n)

            print(f"Об'єднуємо підмасиви: a[{left}:{mid}] ({a[left:mid]})
i a[{mid}:{right}] ({a[mid:right]})")

            c, a_count = merge_iterative(a, left, mid, right)
            comparisons += c
            assignments += a_count

            print(f"Масив після об'єднання: {a}")
            j += 2 * i
        i *= 2
    return a, comparisons, assignments

def merge_iterative(a, left, mid, right):
    comparisons = 0
    assignments = 0
    n1 = mid - left
    n2 = right - mid

    L = a[left:mid]
    R = a[mid:right]
    assignments += n1 + n2

    it1 = 0
    it2 = 0
    k = left
    assignments += 3

    while it1 < n1 and it2 < n2:
        comparisons += 1
        print(f"  Порівняння: {L[it1]} < {R[it2]}")
        if L[it1] <= R[it2]:
            a[k] = L[it1]
            it1 += 1
        else:
```

```
            a[k] = R[it2]
            it2 += 1
        assignments += 1
        k += 1
        assignments += 1

    while it1 < n1:
        a[k] = L[it1]
        it1 += 1
        k += 1
        assignments += 2

    while it2 < n2:
        a[k] = R[it2]
        it2 += 1
        k += 1
        assignments += 2

    return comparisons, assignments

my_list = [79, 97, 82, 18, 20, 2, 88, 61, 17]
print("Початковий масив:", my_list)
sorted_list, comps, assigns = merge_sort_iterative(my_list.copy())
print("Фінальний відсортований список:", sorted_list)
print(f"Загальна кількість порівнянь: {comps}")
print(f"Загальна кількість присвоювань: {assigns}")
```

Лістинг 2.2 – Python-код рекурсивної реалізації алгоритму сортування злиттям.

```
def merge_sort_recursive(arr):
    comparisons = 0
    assignments = 0
    recursive_calls = 1

    if len(arr) <= 1:
        return arr, comparisons, assignments, recursive_calls

    print(f"Розділяємо масив: {arr}")
    mid = len(arr) // 2
    assignments += 1

    left_half, c1, a1, r1 = merge_sort_recursive(arr[:mid])
    right_half, c2, a2, r2 = merge_sort_recursive(arr[mid:])

    comparisons += c1 + c2
    assignments += a1 + a2
    recursive_calls += r1 + r2

    print(f"Зливаємо {left_half} та {right_half}")
    merged_arr, c_merge, a_merge = merge_recursive(left_half, right_half)

    comparisons += c_merge
    assignments += a_merge
    print(f"Злиття завершено. Результат: {merged_arr}")

    return merged_arr, comparisons, assignments, recursive_calls

def merge_recursive(left, right):
    merged_arr = []
    comparisons = 0
    assignments = 0
    i = 0
```

```
        j = 0

        while i < len(left) and j < len(right):
            comparisons += 1
            if left[i] <= right[j]:
                print(f"  Порівняння: {left[i]} <= {right[j]} -> True. Додаємо
{left[i]}")
                merged_arr.append(left[i])
                i += 1
            else:
                print(f"  Порівняння: {left[i]} <= {right[j]} -> False. Додаємо
{right[j]}")
                merged_arr.append(right[j])
                j += 1
            assignments += 1

        while i < len(left):
            merged_arr.append(left[i])
            i += 1
            assignments += 1

        while j < len(right):
            merged_arr.append(right[j])
            j += 1
            assignments += 1

        return merged_arr, comparisons, assignments

    my_list = [79, 97, 82, 18, 20, 2, 88, 61, 17]
    print("Оригінальний список:", my_list)
    sorted_list,          comps,          assigs,          rec_calls          =
merge_sort_recursive(my_list.copy())
    print("Відсортований список:", sorted_list)
    print(f"Загальна кількість порівнянь: {comps}")
    print(f"Загальна кількість присвоювань: {assigs}")
    print(f"Загальна кількість рекурсивних викликів: {rec_calls}")
```

Таблиця 2.2 Результати ітеративної та рекурсивної реалізації алгоритму сортування злиттям

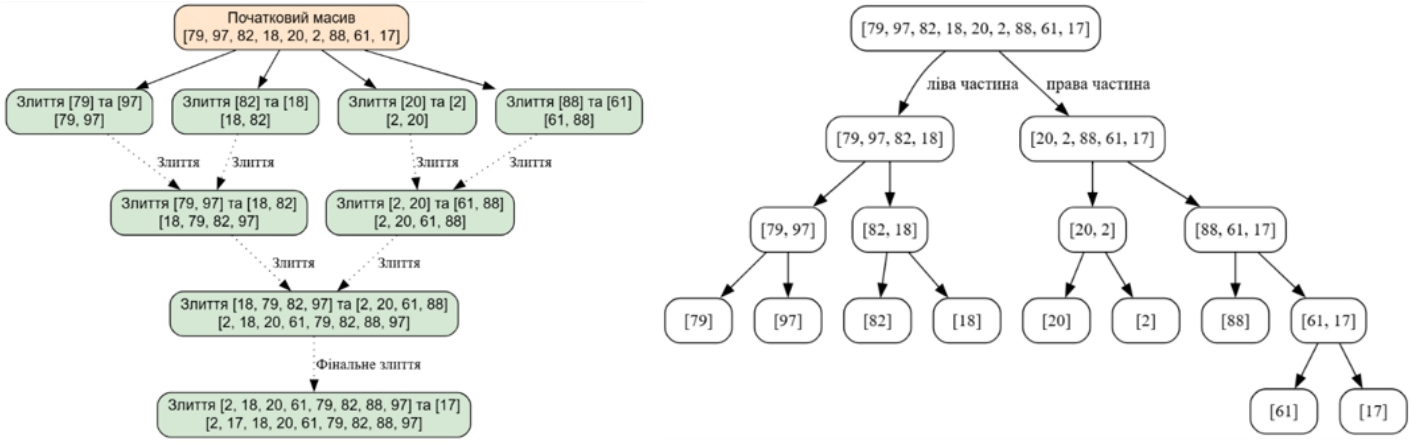
Ітеративний алгоритм	Рекурсивний алгоритм
Оригінальний список: [79, 97, 82, 18, 20, 2, 88, 61, 17]	Оригінальний список: [79, 97, 82, 18, 20, 2, 88, 61, 17]
Відсортований список: [2, 17, 18, 20, 61, 79, 82, 88, 97]	Відсортований список: [2, 17, 18, 20, 61, 79, 82, 88, 97]
Кількість порівнянь: 17	Загальна кількість порівнянь: 20
Кількість порівнянь: 115	Загальна кількість порівнянь: 60
	Загальна кількість рекурсивних викликів: 17

Таблиця 2.3 Результати ітеративної та рекурсивної реалізації алгоритму сортування злиттям

Трасування Python-коду Лістинг 2.1	Трасування Python-коду Лістинг 2.2
---- ІТЕРАТИВНА ВЕРСІЯ ---- Початковий масив: [79, 97, 82, 18, 20, 2, 88, 61, 17] Об'єднуємо підмасиви: a[0:1] ([79]) і a[1:2] ([97])	---- РЕКУРСИВНА ВЕРСІЯ ---- Розділяємо масив: [79, 97, 82, 18, 20, 2, 88, 61, 17] Розділяємо масив: [79, 97, 82, 18] Розділяємо масив: [79, 97]

Порівняння: 79 < 97 Масив після об'єднання: [79, 97, 82, 18, 20, 2, 88, 61, 17] Об'єднуємо підмасиви: a[2:3] ([82]) і a[3:4] ([18]) Порівняння: 82 < 18 Масив після об'єднання: [79, 97, 18, 82, 20, 2, 88, 61, 17] Об'єднуємо підмасиви: a[4:5] ([20]) і a[5:6] ([2]) Порівняння: 20 < 2 Масив після об'єднання: [79, 97, 18, 82, 2, 20, 88, 61, 17] Об'єднуємо підмасиви: a[6:7] ([88]) і a[7:8] ([61]) Порівняння: 88 < 61 Масив після об'єднання: [79, 97, 18, 82, 2, 20, 61, 88, 17] Об'єднуємо підмасиви: a[0:2] ([79, 97]) і a[2:4] ([18, 82]) Порівняння: 79 < 18 Порівняння: 79 < 82 Порівняння: 97 < 82 Масив після об'єднання: [18, 79, 82, 97, 2, 20, 61, 88, 17] Об'єднуємо підмасиви: a[4:6] ([2, 20]) і a[6:8] ([61, 88]) Порівняння: 2 < 61 Порівняння: 20 < 61 Масив після об'єднання: [18, 79, 82, 97, 2, 20, 61, 88, 17] Об'єднуємо підмасиви: a[0:4] ([18, 79, 82, 97]) і a[4:8] ([2, 20, 61, 88]) Порівняння: 18 < 2 Порівняння: 18 < 20 Порівняння: 79 < 20 Порівняння: 79 < 61 Порівняння: 79 < 88 Порівняння: 82 < 88 Порівняння: 97 < 88 Масив після об'єднання: [2, 18, 20, 61, 79, 82, 88, 97, 17] Об'єднуємо підмасиви: a[0:8] ([2, 18, 20, 61, 79, 82, 88, 97]) і a[8:9] ([17]) Порівняння: 2 < 17 Порівняння: 18 < 17 Масив після об'єднання: [2, 17, 18, 20, 61, 79, 82, 88, 97] Фінальний відсортований список: [2, 17, 18, 20, 61, 79, 82, 88, 97] Загальна кількість порівнянь: 17 Загальна кількість присвоювань: 115	Зливаємо [79] та [97] Порівняння: 79 <= 97 -> True. Додаємо 79 Злиття завершено. Результат: [79, 97] Розділяємо масив: [82, 18] Зливаємо [82] та [18] Порівняння: 82 <= 18 -> False. Додаємо 18 Злиття завершено. Результат: [18, 82] Зливаємо [79, 97] та [18, 82] Порівняння: 79 <= 18 -> False. Додаємо 18 Порівняння: 79 <= 82 -> True. Додаємо 79 Порівняння: 97 <= 82 -> False. Додаємо 82 Злиття завершено. Результат: [18, 79, 82, 97] Розділяємо масив: [20, 2, 88, 61, 17] Розділяємо масив: [20, 2] Зливаємо [20] та [2] Порівняння: 20 <= 2 -> False. Додаємо 2 Злиття завершено. Результат: [2, 20] Розділяємо масив: [88, 61, 17] Розділяємо масив: [88] Розділяємо масив: [61, 17] Зливаємо [61] та [17] Порівняння: 61 <= 17 -> False. Додаємо 17 Злиття завершено. Результат: [17, 61] Зливаємо [88] та [17, 61] Порівняння: 88 <= 17 -> False. Додаємо 17 Порівняння: 88 <= 61 -> False. Додаємо 61 Злиття завершено. Результат: [17, 61, 88] Зливаємо [2, 20] та [17, 61, 88] Порівняння: 2 <= 17 -> True. Додаємо 2 Порівняння: 20 <= 17 -> False. Додаємо 17 Порівняння: 20 <= 61 -> True. Додаємо 20 Злиття завершено. Результат: [2, 17, 20, 61, 88] Зливаємо [18, 79, 82, 97] та [2, 17, 20, 61, 88] Порівняння: 18 <= 2 -> False. Додаємо 2 Порівняння: 18 <= 17 -> False. Додаємо 17 Порівняння: 18 <= 20 -> True. Додаємо 18

	Порівняння: 79 <= 20 -> False. Додаємо 20 Порівняння: 79 <= 61 -> False. Додаємо 61 Порівняння: 79 <= 88 -> True. Додаємо 79 Порівняння: 82 <= 88 -> True. Додаємо 82 Порівняння: 97 <= 88 -> False. Додаємо 88 Злиття завершено. Результат: [2, 17, 18, 20, 61, 79, 82, 88, 97] Фінальний відсортований список: [2, 17, 18, 20, 61, 79, 82, 88, 97] Загальна кількість порівнянь: 20 Загальна кількість присвоювань: 60 Загальна кількість рекурсивних викликів: 17



а б

Рисунок 2.1 – Результати візуалізації трасування ітеративної та рекурсивної версій Python-коду алгоритму сортування злиттям.

На рисунку 2.1 показано візуалізацію процесів сортування злиттям для масиву з 9 елементів двома способами: ітеративним (а) та рекурсивним (б).
У рекурсивній версії (Рис. 2.1, б) алгоритм працює за принципом "зверху вниз". Відповідно до псевдокоду (Табл. 2.1) та трасування (Табл. 2.3), масив спочатку рекурсивно ділиться навпіл до базового випадку (один елемент), а лише після цього починається фаза злиття.
В ітеративній версії (Рис. 2.1, а) алгоритм працює за принципом "знизу вгору". Рекурсія відсутня. Відповідно до циклу for i=1 to n, i*=2 у Лістингу 2.1, масив одразу сприймається як набір блоків розміром 1. Алгоритм поступово зливає сусідні блоки попарно: спочатку по 1 елементу, потім по 2, по 4 і по 8, поки весь масив не буде відсортовано. Останній непарний елемент 17 приєднується до загального відсортованого масиву на фінальному кроці, що підтверджується даними трасування.

2. Швидке сортування

Таблиця 2.4 – Приклади псевдокоду рекурсивної процедури та процедури розбиття алгоритму швидкого сортування за схемою Хоара

Рекурсивна процедура quicksort quicksort(a, l, r)	Процедура розбиття partition

def quicksort(a, l, r):
if l < r:
q = partition(a, l, r)
quicksort(a, l, q) #
Рекурсивний виклик для лівого підмасиву
quicksort(a, q + 1, r) #
Рекурсивний виклик для правого підмасиву
	def partition(a, l, r):
	pivot = a[l]
	i = l - 1
	j = r + 1
	while True:
	i += 1
	while a[i] < pivot:
	i += 1
	j -= 1
	while a[j] > pivot:
	j -= 1
	if i >= j:
	return j
	a[i], a[j] = a[j], a[i]

Лістинг 2.3 – Python-код рекурсивної реалізації алгоритму швидкого сортування за схемою Хоара

```
def quicksort(a, l, r):
    comparisons = 0
    assignments = 0
    recursive_calls = 1

    if l < r:
        print(f"Quicksort виклик: масив = {a}, l={l}, r={r}")
        q, c1, a1 = partition(a, l, r)
        comparisons += c1
        assignments += a1

        c2, a2, r2 = quicksort(a, l, q)
        c3, a3, r3 = quicksort(a, q + 1, r)

        comparisons += c2 + c3
        assignments += a2 + a3
        recursive_calls += r2 + r3

        print(f"Після Quicksort для l={l}, r={r}: масив = {a}")
    return comparisons, assignments, recursive_calls

def partition(a, l, r):
    comparisons = 0
    assignments = 0

    pivot = a[l]
    assignments += 1
    print(f"  Вибираємо опорний елемент (pivot): {pivot}")

    i = l - 1
    j = r + 1
```

```
    assignments += 2

    while True:
        i += 1
        assignments += 1
        while a[i] < pivot:
            comparisons += 1
            i += 1
            assignments += 1
        comparisons += 1

        j -= 1
        assignments += 1
        while a[j] > pivot:
            comparisons += 1
            j -= 1
            assignments += 1
        comparisons += 1

        comparisons += 1
        if i >= j:
            print(f"  Індекси перетнулися (i={i}, j={j}). Поділ завершено.
Повертаємо j={j}.")
            return j, comparisons, assignments

        a[i], a[j] = a[j], a[i]
        assignments += 3
        print(f"  Обмінюємо a[{i}] ({a[j]}) і a[{j}] ({a[i]}). Масив: {a}")

my_list = [79, 97, 82, 18, 20, 2, 88, 61, 17]
original_list = my_list.copy()
print("Оригінальний список:", original_list)

total_comparisons,     total_assignments,     total_recursive_calls     =
quicksort(my_list, 0, len(my_list) - 1)

print("Відсортований список:", my_list)
print(f"Загальна кількість порівнянь: {total_comparisons}")
print(f"Загальна кількість присвоювань: {total_assignments}")
print(f"Загальна кількість рекурсивних викликів: {total_recursive_calls}")
```

Таблиця 2.5 Результати трасування алгоритму швидкого сортування

Трасування Python-коду (Лістинг 3)
--- Quicksort за схемою Хоара з трасуванням --- Оригінальний список: [79, 97, 82, 18, 20, 2, 88, 61, 17] Quicksort виклик: масив = [79, 97, 82, 18, 20, 2, 88, 61, 17], l=0, r=8 Вибираємо опорний елемент (pivot): 79 Обмінюємо a[0] (17) і a[8] (79). Масив: [17, 97, 82, 18, 20, 2, 88, 61, 79] Обмінюємо a[1] (61) і a[7] (97). Масив: [17, 61, 82, 18, 20, 2, 88, 97, 79] Обмінюємо a[2] (2) і a[5] (82). Масив: [17, 61, 2, 18, 20, 82, 88, 97, 79] Індекси перетнулися (i=5, j=4). Поділ завершено. Повертаємо j=4. Quicksort виклик: масив = [17, 61, 2, 18, 20, 82, 88, 97, 79], l=0, r=4 Вибираємо опорний елемент (pivot): 17 Обмінюємо a[0] (2) і a[2] (17). Масив: [2, 61, 17, 18, 20, 82, 88, 97, 79] Індекси перетнулися (i=1, j=0). Поділ завершено. Повертаємо j=0. Quicksort виклик: масив = [2, 61, 17, 18, 20, 82, 88, 97, 79], l=1, r=4 Вибираємо опорний елемент (pivot): 61 Обмінюємо a[1] (20) і a[4] (61). Масив: [2, 20, 17, 18, 61, 82, 88, 97, 79] Індекси перетнулися (i=4, j=3). Поділ завершено. Повертаємо j=3.

```
Quicksort виклик: масив = [2, 20, 17, 18, 61, 82, 88, 97, 79], l=1, r=3
Вибираємо опорний елемент (pivot): 20
Обмінюємо a[1] (18) і a[3] (20). Масив: [2, 18, 17, 20, 61, 82, 88, 97, 79]
Індекси перетнулися (i=3, j=2). Поділ завершено. Повертаємо j=2.
Quicksort виклик: масив = [2, 18, 17, 20, 61, 82, 88, 97, 79], l=1, r=2
Вибираємо опорний елемент (pivot): 18
Обмінюємо a[1] (17) і a[2] (18). Масив: [2, 17, 18, 20, 61, 82, 88, 97, 79]
Індекси перетнулися (i=2, j=1). Поділ завершено. Повертаємо j=1.
Quicksort виклик: масив = [2, 17, 18, 20, 61, 82, 88, 97, 79], l=5, r=8
Вибираємо опорний елемент (pivot): 82
Обмінюємо a[5] (79) і a[8] (82). Масив: [2, 17, 18, 20, 61, 79, 88, 97, 82]
Індекси перетнулися (i=6, j=5). Поділ завершено. Повертаємо j=5.
Quicksort виклик: масив = [2, 17, 18, 20, 61, 79, 88, 97, 82], l=6, r=8
Вибираємо опорний елемент (pivot): 88
Обмінюємо a[6] (82) і a[8] (88). Масив: [2, 17, 18, 20, 61, 79, 82, 97, 88]
Індекси перетнулися (i=7, j=6). Поділ завершено. Повертаємо j=6.
Quicksort виклик: масив = [2, 17, 18, 20, 61, 79, 82, 97, 88], l=7, r=8
Вибираємо опорний елемент (pivot): 97
Обмінюємо a[7] (88) і a[8] (97). Масив: [2, 17, 18, 20, 61, 79, 82, 88, 97]
Індекси перетнулися (i=8, j=7). Поділ завершено. Повертаємо j=7.

Фінальний відсортований список: [2, 17, 18, 20, 61, 79, 82, 88, 97]
Загальна кількість порівнянь: 45
Загальна кількість присвоювань: 76
Загальна кількість рекурсивних викликів: 17
```

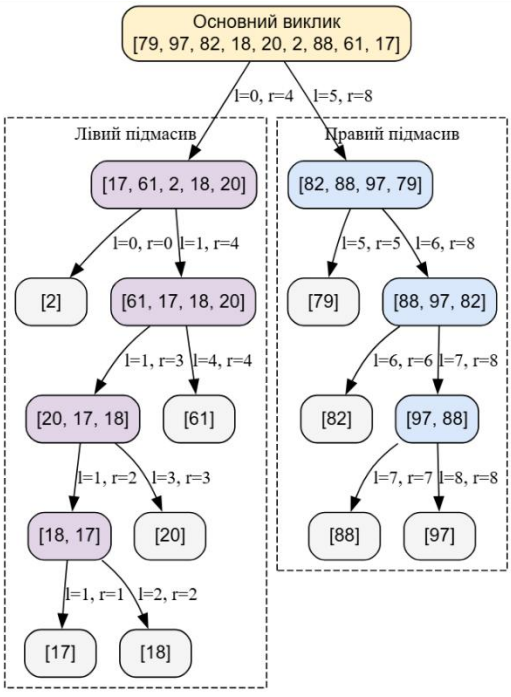


Рисунок 2.2 Результати візуалізації трасування Python-коду рекурсивного алгоритму швидкого сортування

На рисунку 2.2 показано візуалізацію трасування рекурсивного алгоритму швидкого сортування за схемою Хоара. Відповідно до псевдокоду (Табл. 2.4) та Python-коду (Лістинг 2.3), масив розділяється на дві частини відносно опорного елемента, яким завжди обирається перший елемент підмасиву. З графа та Таблиці 2.5 видно, що на першому кроці масив розбився на лівий [17, 61, 2, 18, 20] та правий [82, 88, 97, 79] підмасиви. На відміну від сортування злиттям, етап явного об'єднання тут відсутній, оскільки масив сортується in-place безпосередньо під час розбиття. Алгоритм завершується, коли всі рекурсивні виклики досягають базового випадку ($l \geq r$), яких для даного масиву було зафіксовано 17

3. Аналіз обчислювальної складності алгоритмів сортування злиттям та швидкого сортування

Таблиця 1.3 Порівняння обчислювальної складності алгоритмів сортування вибором та вставками

Назва алгоритму	Теоретичні відомості		Результати експерименту n=9 A=[79_0, 97_1, 82_2, 18_3, 20_4, 2_5, 88_6, 61_7, 17_8]		
	Часова складність	Пам’ять	порівняння	присвоювання	виклики
Merge Sort (Ітеративний)	*O(n log n),*	*O(n) додаткова*	17	115	0
Merge Sort (Рекурсивний)	*O(n log n) (Best)* *O(n^2) (Worst, Average)*	*O(n) додаткова +* *O(log n) стек*	20	60	17
Quick Sort	*O(n log n) (Best, Average)* *O(n^2) (Worst)*	*O(1) in-place +* *O(log n) стек*	45	76	17

Висновки: Під час виконання другої лабораторної роботи я набув практичних навичок із проєктування, реалізації, тестування та аналізу логарифмічних алгоритмів сортування: злиттям (Merge sort, ітеративна та рекурсивна версії) та швидкого сортування (Quick sort за схемою Хоара).

Порівняльний аналіз показав, що рекурсивне сортування злиттям для мого масиву з 9 елементів виконало найменшу кількість порівнянь та присвоювань (20 та 60 відповідно), але воно вимагає додаткової оперативної пам'яті $O(n)$ для

допоміжних масивів та створює навантаження на стек (17 рекурсивних викликів). Ітеративна версія злиття повністю усунула навантаження на стек, проте збільшила кількість операцій копіювання. Швидке сортування виконало найбільше базових операцій (45 порівнянь, 76 присвоювань), але його перевагою є виконання сортування на місці (in-place) без використання додаткової пам'яті для масивів.

Посилання на GitHub: https://github.com/9998920/algorithms-labs